SDL CORE 10 v0.6

Independent Python Wi-Fi Gateway | Operator and Teacher Guide
A working simulation, real TCP/IP communication, and a browser laboratory - documented with explicit verification boundaries.

What you receive

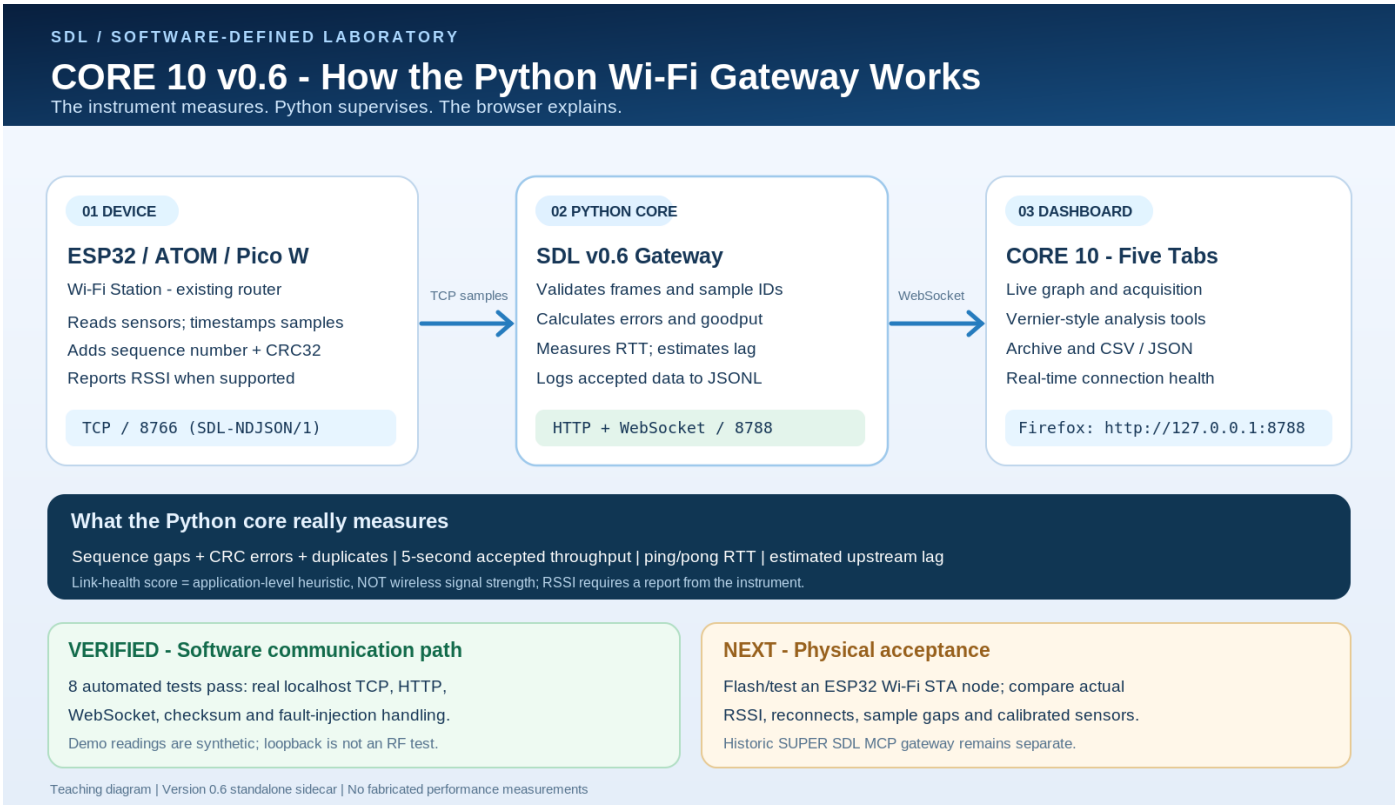
- **CORE 10 graphical analysis:** five-tab dashboard, ten simulations and eight extensions, analysis, and archive.
- **Python gateway:** TCP node acquisition, CRC32/sequence validation, rolling throughput, RTT, estimated one-way lag, live WebSocket and JSONL logs.
- **Standalone synthetic instrument:** a separate TCP server so the full communications path works on one Windows PC.
- **Hardware example:** Arduino ESP32 Wi-Fi STA sketch for a future physical node test (not hardware verified).
- **Testing:** eight passing automated tests, including fault injection; full results in the included test report.

Start the demo on Windows

- 1. Extract the ZIP to a writable folder; keep the directory structure.
- 2. Install Python 3.10+; verify with `py -3 --version`.
- 3. Double-click `START_DEMO.bat`. Wait for the browser at `http://127.0.0.1:8788/`.
- 4. In **Connections**, select **Python SDL Wi-Fi Gateway - live** and click **Connect source**.
- 5. In **Live**, choose **Galileo Measurement Lab**, then **Start run**.
- 6. Inspect metrics, Analyze, Stop, Save and export. Press Ctrl+C in the console to stop the server.

Important: Readings in the default demonstration are synthetic even though they are carried across genuine loopback TCP sockets and WebSocket frames.

Architecture - what talks to what



The same UI supports local simulation, a browser WebSocket link and, when implemented and tested, physical network instruments. The legacy SUPER SDL v0.7 MCP gateway remains independent; this download does not install or modify it.

Why two links?

- **TCP port 8766:** device-to-Python. A persistent, ordered stream carries newline-delimited JSON envelopes with sequence IDs and checksums.
- **HTTP/WebSocket port 8788:** Python-to-browser. The dashboard receives measured values plus once-per-second diagnostics. Firefox works with WebSocket.
- **Historical ports preserved:** MCP 8000, old SDL ASCII device 8765, and optional Traffic Cop 8787 are not overwritten by v0.6.

What the four network numbers mean

Readout	What Python really knows
Link health	Derived 0-100 application score: recent gaps, CRC faults, duplicates, reordered frames, parse errors and staleness. Not Wi-Fi RF strength.
Data integrity	Checks CRC32 and ordered sequence IDs; displays missing/duplicate/error counters. Sample gaps do not directly equal radio packet loss.
Throughput	Accepted payload bytes per second (5-second window) and good samples/s; excludes network overhead and retransmits.
Lag / RTT	Python ping-pong round-trip and estimated upstream sample lag from device monotonic timestamps. Clock uncertainty is approximately half the RTT.
RSSI	Only available when the instrument itself supplies a radio RSSI measurement. TCP cannot reveal it.

Prove the detection system by causing failures

Run this instead of START_DEMO.bat (from a terminal opened in the extracted folder):

```
py -3 launch.py --demo --drop-every 5 --bad-crc-every 7 --duplicate-every 4 --delay-ms 80 --open-browser
```

- The simulator deliberately omits a sample, corrupts an envelope CRC, repeats a frame and adds node-side delay.
- Watch the Connections panel. Errors are cumulative, while the health score uses the most recent 30 seconds.
- Restart cleanly to see the score recover. A green score is never proof of sensor calibration or radio quality.

Operate real Wi-Fi hardware safely

The physical acceptance path

- 1. First pass the synthetic demo and eight Python tests.
- 2. Open the reference Arduino sketch in `arduino/SDL_WiFi_Node_Reference.ino`. It targets an ESP32 with Arduino Wi-Fi support.
- 3. Insert your own SSID and password; never publish the modified credentials. Flash the board and read its DHCP address on Serial Monitor.
- 4. Run `py -3 launch.py --node-host DEVICE_IP --node-port 8766 --open-browser`.
- 5. Use the Python Gateway plugin in Connections. Compare sample rate, gaps, CRC faults, RTT and node-reported RSSI.
- 6. Verify the physical sensor against a known reference. Then incorporate the ATOM weight-I2C water-clock module only with correct firmware and wiring.

Electrical caution: the illustrative GPIO32 analog input is limited to 3.3 V and conflicts with Grove SCL on the original ATOM Matrix. The Arduino sample does not yet drive the weight-I2C module at 0x26. No real ESP32 board was flashed or tested in this verification.

System safety

- The web server binds to 127.0.0.1 by default. No login, TLS or internet exposure; do not open or port-forward gateway ports.
- Demo experiment selection is allowlisted. The gateway intentionally does not execute physical actuator commands.
- CRC32 checks accidental corruption; it does not authenticate the sender. Wi-Fi equipment should use a trusted local network.

If something does not connect

- Run `py -3 -m unittest discover -s tests -v` and first confirm clean local tests.
- Check the Python console for the HTTP address and TCP node connection, plus `/api/status` in Firefox.
- In Connections, use the exact URL `ws://127.0.0.1:8788/ws/telemetry`. Click Connect before starting the Live run.
- If using hardware, ensure PC and node are on the same LAN and port 8766 is reachable; try the standalone demo to isolate the issue.

Acceptance evidence and limitations

Passed: eight independent Python automated unit/integration tests on November 8, 2026. Coverage includes CRC rejection, sequence integrity, RSSI provenance, health recovery, actual loopback TCP node streaming, HTTP status, RFC6455 WebSocket metrics/samples, and deliberate fault detection.

Not yet demonstrated: real ESP32/ATOM Wi-Fi radio link, real physical sensors, Arduino sketch compilation/flashing on your hardware, and manual Firefox/Windows walkthrough. Browser automation in this environment is blocked by administrator policy for local URLs; Python-level WebSocket tests ran successfully.

Not included: old SDL ASCII adapter, MCP gateway integration, TLS/authentication, multi-node clock synchronization, certified metrology or high-rate acquisition guarantees.

Core educational principle

The instrument produces an observation. Python protects and audits its provenance. The browser offers ways to challenge the observation with mathematics. The measured points must always remain distinguishable from the model and from synthetic examples.

Package map

File / folder	Use
START_DEMO.bat / launch.py	Start full simulation and gateway on one computer.
web/	Five-tab dashboard and communications plugin.
gateway/	TCP simulator, validation, WebSocket, diagnostics.
tests/	Repeatable automated Python checks.
arduino/	ESP32 Wi-Fi STA example; hardware-unverified.
docs/	Guided lesson, metrics reference, protocol, test report and infographic.

Complete source documentation is included in the downloadable ZIP. This PDF is an introductory lesson; it is not a hardware validation certificate.